

Trabajo Práctico 2

I. Introducción a Spring Data JPA

1. Spring Data JPA y su lugar en el ecosistema

Inciso 1

¿Qué es Spring Data JPA? ¿Qué problema resuelve respecto de usar Hibernate directamente? Describir dos situaciones del proyecto donde Spring Data JPA simplifica código que en la Práctica 1 requería implementación manual

Respuesta:

Spring Data JPA es una capa de **abstracción** sobre el ORM (Hibernate) que forma parte del ecosistema Spring. No es un ORM en sí mismo, sino un framework diseñado para simplificar significativamente la construcción de la capa de acceso a datos.

Problemas que resuelve respecto de Hibernate directo

El principal problema que resuelve es la eliminación del **código boilerplate** (código repetitivo y manual). Mientras que Hibernate directo requiere que el desarrollador gestione la lógica técnica de persistencia, Spring Data JPA ofrece las siguientes ventajas:

- **Implementación automática de repositorios:** Evita tener que escribir clases concretas para cada repositorio. El framework genera la implementación en tiempo de ejecución (runtime) a través de proxies dinámicos.
- **Gestión transparente de la Sesión:** Elimina la necesidad de inyectar la `SessionFactory` y llamar manualmente a `getCurrentSession()` en cada método del repositorio.
- **Abstracción de operaciones comunes:** Provee automáticamente operaciones estándar de persistencia (CRUD) y capacidades avanzadas como paginación y ordenamiento nativo sin código adicional.

Situaciones del proyecto donde simplifica el código

Al migrar la Práctica 1 (Hibernate directo) a la Práctica 2 (Spring Data JPA), se identifican dos situaciones clave de simplificación:

① Definición de Repositorios mediante Interfaces:

- **En la Práctica 1:** Se debía implementar una clase concreta por cada repositorio (ej. `PurchaseRepositoryImpl`), inyectar la `SessionFactory`, obtener la sesión activa y codificar manualmente métodos como `save()`, `buscarPorId()` o `eliminar()`.
- **Con Spring Data JPA:** El repositorio pasa a ser una **interfaz** que extiende de `CrudRepository<Entidad, ID>`. Spring Data JPA implementa automáticamente todos los métodos básicos de persistencia, eliminando decenas de líneas de código manual.

② Generación de Consultas mediante Query Methods:

- **En la Práctica 1:** Para realizar una consulta simple (como buscar un usuario por su email), era necesario escribir manualmente la consulta en **HQL**, crear el objeto **Query**, setear los parámetros y ejecutar la sesión.
- **Con Spring Data JPA:** Se utilizan los **Query Methods**, donde el framework deriva la consulta SQL automáticamente a partir del nombre del método definido en la interfaz (ej. `findByEmail(String email)`). Esto mejora la legibilidad y reduce el acoplamiento a la infraestructura de persistencia.

Inciso 2

Spring Data JPA no es un ORM sino una capa de abstracción sobre el ORM. Explicar la diferencia: ¿qué hace Spring Data JPA? ¿Qué sigue haciendo Hibernate internamente?

Respuesta:

Spring Data JPA no es un motor de persistencia (ORM) en sí mismo, sino una **capa de abstracción** diseñada para simplificar el uso de JPA y Hibernate, eliminando la necesidad de escribir código repetitivo o "boilerplate".

A continuación se detalla la división de responsabilidades entre ambas tecnologías:

¿Qué hace Spring Data JPA?

Su función principal es facilitar la construcción de la capa de acceso a datos (Repositorios). Entre sus tareas específicas se encuentran:

- **Generación de Repositorios en Runtime:** El desarrollador solo define **interfaces** (que extienden de `CrudRepository` o `JpaRepository`) y el framework genera la implementación concreta automáticamente en tiempo de ejecución mediante **proxies dinámicos**.
- **Implementación Automática de CRUD:** Provee de forma nativa los métodos estándar como `save()`, `findById()`, `findAll()` y `deleteById()`, evitando que el programador tenga que codificarlos manualmente con la `Session` de Hibernate.
- **Query Methods (Consultas por nombre):** Es capaz de **derivar consultas SQL** automáticamente a partir del nombre de un método definido en la interfaz (por ejemplo, `findByEmail`), ahorrando la escritura de HQL o JPQL para operaciones simples.
- **Gestión Declarativa de Transacciones:** Permite propagar y controlar los límites de la unidad de trabajo mediante la anotación `@Transactional`.
- **Soporte de Paginación y Ordenamiento:** Ofrece abstracciones como `Pageable` y `Sort` para manejar grandes volúmenes de datos de forma nativa.

¿Qué sigue haciendo Hibernate internamente?

Hibernate continúa siendo el **motor de persistencia (ORM)** que realiza el "trabajo sucio" de bajo nivel. Aunque Spring Data JPA lo "envuelve", Hibernate sigue siendo responsable de:

- **Gestionar el Contexto de Persistencia:** Mantiene el control del **ciclo de vida de las entidades** (estados *managed*, *detached*, *removed*, etc.) y asegura que los cambios en los objetos se sincronicen con la base de datos.
- **Mapeo Objeto-Relacional:** Es el encargado de interpretar las anotaciones (`@Entity`, `@Table`, `@OneToMany`) para saber cómo transformar una clase en una tabla y viceversa.
- **Traducción a SQL:** Spring Data JPA le entrega una consulta en JPQL o un criterio de búsqueda, y Hibernate es quien la **traduce al dialecto SQL específico** del motor de base de datos utilizado (Oracle, MySQL, etc.).
- **Mapeo de ResultSet:** Transforma las filas y columnas devueltas por la base de datos en **objetos Java "reales"**.
- **Gestión de Proxies y Lazy Loading:** Controla cuándo recuperar un colaborador de una entidad (carga diferida) para optimizar el rendimiento.

En resumen, mientras **Spring Data JPA** se encarga de que escribas **menos código** para definir cómo acceder a los datos, **Hibernate** se encarga de la **lógica técnica** de cómo esos datos se mueven entre el mundo de los objetos y el mundo relacional.

Inciso 3

La siguiente tabla lista tareas relacionadas con la persistencia. Marcar con una X la columna de la tecnología que resuelve ese problema en la nueva implementación con Spring Data JPA:

Tarea	JDBC	Hibernate	Spring Data JPA	Justificación
Abrir y cerrar la conexión a la base de datos		X		En JDBC puro , el desarrollador es el responsable directo de abrir y cerrar las conexiones físicas mediante código manual. Hibernate y Spring Data JPA resuelven esto automatizándolo a través de la sesión y el pool.
Implementar <code>save()</code> , <code>findById()</code> y <code>deleteById()</code>			X	Spring Data JPA provee automáticamente la implementación de estas operaciones CRUD estándar, eliminando el código manual ("boilerplate") que se requería en la Práctica 1.
Gestionar el ciclo de vida de las entidades (<code>@Entity</code>)		X		Aunque Spring Data JPA lo abstrae, es Hibernate el que sigue siendo responsable internamente de gestionar el Contexto de Persistencia y los estados de las

Tarea	JDBC	Hibernate	Spring Data JPA	Justificación
				entidades (<i>managed</i> , <i>detached</i> , <i>removed</i>).
Derivar una consulta a partir del nombre del método			X	Esta es una funcionalidad exclusiva de Spring Data JPA llamada Query Methods , donde el framework interpreta nombres como <code>findByEmail</code> para generar la consulta.
Manejar el pool de conexiones		X		Hibernate se encarga de administrar el pool de conexiones a la base de datos para optimizar este recurso costoso, a menudo integrándose con librerías externas.
Propagar transacciones con <code>@Transactional</code>			X	La gestión declarativa de los límites de la unidad de trabajo mediante la anotación @Transactional es una capacidad proporcionada por el ecosistema de Spring.
Generar la implementación del repositorio en runtime			X	Spring Data JPA genera las implementaciones concretas de tus interfaces de repositorio en tiempo de ejecución utilizando proxies dinámicos .
Mapear ResultSet a objetos Java		X		El "trabajo sucio" de transformar las filas y columnas del SQL en objetos Java reales lo sigue realizando Hibernate internamente.
Proveer soporte nativo de paginación (Pageable)			X	Spring Data JPA incorpora soporte nativo para paginar y ordenar resultados mediante las abstracciones Pageable y <code>Sort</code> .

2. Configuración del proyecto

Inciso 4

Listar los cambios que deben realizarse en el proyecto de la Práctica 1 para incorporar Spring Data JPA.

Respuesta:

- ① Cambios en la Configuración e Infraestructura

- **Gestión de Dependencias:** Se deben actualizar las dependencias en el archivo **pom.xml** para incluir los starters de Spring Data JPA.
- **Reemplazo de la Configuración de Persistencia:** La clase **HibernateConfiguration** (usada en la Práctica 1 para obtener la **SessionFactory**) debe ser reemplazada por la clase **SpringDataConfiguration**.
- **Externalización de Propiedades:** La configuración técnica de la base de datos (URL, credenciales, dialecto y estrategia de generación de tablas) se traslada de clases Java al archivo **application.properties** de Spring Boot.
- **Actualización de Clases de Apoyo:** Se deben modificar la clase **AppConfig**, el inicializador de la base de datos **DBInitializer** y los archivos de testing para que sean compatibles con el nuevo motor.

① Rediseño de la Capa de Repositorios

- **De Clases a Interfaces:** Los repositorios ya no son clases concretas (implementaciones manuales), sino **interfaces**.
- **Herencia de CrudRepository:** Cada interfaz de repositorio (ej. **RouteRepository**) debe extender de **CrudRepository<Entidad, ID>**, especificando el tipo de entidad y su clave primaria.
- **Eliminación de Código "Boilerplate":** Se deben eliminar las implementaciones manuales de métodos como **save()**, **findById()** y **delete()**, ya que Spring Data JPA las genera automáticamente en tiempo de ejecución.

① Migración de Consultas

- **Adopción de Query Methods:** Las consultas simples (como buscar por nombre o email) que antes requerían HQL manual ahora se resuelven mediante **métodos por nombre** definidos en la interfaz (ej. **findByUsername**).
- **Uso de @Query:** Para consultas complejas que involucren JOINS o agrupaciones, se debe utilizar la anotación **@Query** directamente sobre el método en la interfaz, eliminando la necesidad de manejar el objeto **Query** de Hibernate manualmente.

① Adaptación de la Capa de Servicio

- **Inyección de Repositorios:** Se debe crear una nueva implementación del servicio (ej. **ToursServiceImpl**) que inyecte las **interfaces de los repositorios** en lugar de interactuar con la **SessionFactory**.
- **Actualización de Llamadas:** Las llamadas directas a la sesión de Hibernate (ej. **session.save** o **session.createQuery**) dentro de los servicios deben ser reemplazadas por los métodos equivalentes provistos por el nuevo repositorio.

Lo que NO debe cambiarse

- **Mapeo de Entidades:** El mapeo ya implementado (anotaciones **@Entity**, **@Table**, relaciones **@OneToMany** / **@ManyToMany**, estrategias de herencia, cascadas y **FetchType**)

no debe modificarse, ya que Spring Data JPA utiliza la misma infraestructura de Hibernate para estas definiciones.

Inciso 5

En la Práctica 1 se configuraba Hibernate mediante una clase de configuración Java para obtener la SessionFactory. ¿Cómo se reemplaza esta configuración usando application.properties en Spring Boot?

1. Cambio de Paradigma: De Código a Propiedades

En la Práctica 1, se utilizaba una clase de configuración Java (usualmente llamada `HibernateConfiguration`) para instanciar manualmente la **SessionFactory** y definir los parámetros técnicos. En Spring Boot, esta configuración se externaliza al archivo `application.properties`, lo que permite que el framework gestione la infraestructura de persistencia de forma automática.

2. Mapeo de Propiedades Técnicas

Los valores que antes se codeaban en la clase Java ahora se definen mediante las siguientes propiedades estándar:

- **Datos de Conexión (Datasource):** Se configuran mediante `spring.datasource.url`, `spring.datasource.username`, `spring.datasource.password` y `spring.datasource.driver-class-name`.
- **Dialecto de Hibernate:** Se especifica a través de `spring.jpa.properties.hibernate.dialect`, indicando al ORM cómo comunicarse con el motor de base de datos específico (ej. MySQL).
- **Estrategia del Esquema:** La propiedad `spring.jpa.hibernate.ddl-auto` reemplaza la lógica manual para controlar si Hibernate debe **crear, actualizar o validar** las tablas de la base de datos automáticamente al iniciar la aplicación.

3. Reemplazo de la Clase de Configuración

A nivel de código, la clase `HibernateConfiguration` de la Práctica 1 es eliminada y reemplazada por la clase **SpringDataConfiguration**. Esta nueva clase trabaja en conjunto con las propiedades definidas en `application.properties` para inicializar el contexto de Spring Data JPA sin necesidad de que el desarrollador gestione directamente la creación de la `SessionFactory`.

Inciso 6

Describir las propiedades más relevantes de Spring Data JPA que deben configurarse en application.properties para este proyecto. Incluir al menos: datasource (url, username, password, driver), dialecto de Hibernate, y la propiedad que controla si Hibernate debe crear, actualizar o validar el esquema. ¿Cuál de estos valores conviene usar durante el desarrollo?

Respuesta:

En la migración a **Spring Data JPA**, la configuración técnica de la persistencia se traslada al archivo **application.properties**, permitiendo que el framework gestione la infraestructura de forma automática y centralizada.

A continuación, se describen las propiedades más relevantes para este proyecto:

1. Configuración del Datasource (Conexión)

Estas propiedades definen cómo la aplicación se comunica físicamente con la base de datos:

- **spring.datasource.url**: Especifica la dirección de la base de datos (ej. `jdbc:mysql://localhost:3306/tours_db`).
- **spring.datasource.username**: El nombre de usuario para acceder al motor de base de datos.
- **spring.datasource.password**: La contraseña correspondiente al usuario.
- **spring.datasource.driver-class-name**: El nombre de la clase del driver JDBC específico del motor utilizado (ej. `com.mysql.cj.jdbc.Driver`).

2. Dialecto de Hibernate

- **spring.jpa.properties.hibernate.dialect**: Indica a Hibernate el protocolo SQL específico del motor que se está utilizando (ej. `org.hibernate.dialect.MySQLDialect`). Esto es crucial para que el ORM traduzca correctamente las consultas HQL/JPQL al SQL nativo del motor.

3. Control del Esquema (DDL-Auto)

La propiedad **spring.jpa.hibernate.ddl-auto** controla si Hibernate debe intervenir en la estructura de las tablas al iniciar la aplicación. Los valores más comunes son:

- **update**: Actualiza el esquema existente añadiendo nuevas columnas o tablas sin borrar los datos previos.
- **create**: Crea las tablas desde cero cada vez que inicia la aplicación, borrando cualquier dato anterior.
- **create-drop**: Similar a *create*, pero además borra las tablas al cerrar la aplicación.
- **validate**: Solo verifica que el esquema de la base de datos coincida con el mapeo de las entidades, sin realizar cambios.
- **none**: No realiza ninguna acción sobre el esquema.

¿Cuál conviene usar durante el desarrollo?

Durante la etapa de **desarrollo**, lo más conveniente suele ser utilizar el valor **update**. Esto permite que el esquema de la base de datos evolucione a la par de los cambios en el modelo de objetos (como agregar nuevos atributos a `User`) sin perder los datos de prueba ya cargados.

Adicionalmente, se recomienda activar la propiedad **spring.jpa.show-sql=true** durante esta etapa para poder visualizar en la consola las sentencias SQL que Hibernate genera automáticamente y así facilitar el proceso de depuración.

3. La interfaz **CrudRepository** y la jerarquía de repositorios

Inciso 7

¿Qué es **CrudRepository**? ¿De qué interfaz hereda y qué operaciones provee automáticamente?

Respuesta:

CrudRepository es una **interfaz** fundamental de Spring Data JPA que actúa como el punto de acceso para gestionar la persistencia de datos de una entidad específica. Su propósito principal es eliminar el "**código boilerplate**" (código repetitivo) al permitir que el framework genere automáticamente la implementación del repositorio en tiempo de ejecución a través de proxies dinámicos.

En cuanto a su jerarquía, **CrudRepository** hereda de la interfaz **Repository**, la cual es la interfaz marcadora de más alto nivel dentro del ecosistema de Spring Data.

Esta interfaz provee de forma automática las operaciones estándar de **CRUD** (Create, Read, Update, Delete) sin necesidad de codificación manual. Las operaciones principales que incorpora son:

- **save(S entity)**: Utilizada tanto para crear nuevas instancias como para actualizar las existentes.
- **findById(ID id)**: Permite recuperar una entidad única mediante su clave primaria.
- **findAll()**: Retorna todas las instancias de la entidad almacenadas en el repositorio.
- **deleteById(ID id)**: Elimina una instancia específica de la base de datos.
- **count()**: Devuelve la cantidad total de registros persistidos para esa entidad.
- **existsById(ID id)**: Verifica la existencia de un objeto basándose en su identificador.

Para su utilización en el proyecto, el desarrollador solo debe crear una interfaz que extienda de **CrudRepository**, especificando el **tipo de entidad** y el **tipo de su identificador** (por ejemplo, **CrudRepository<Route, Long>**).

Inciso 8

¿Que agrega cada nivel de la jerarquía respecto del anterior? Describir brevemente las diferencias entre **CrudRepository**, **PagingAndSortingRepository** y **JpaRepository**, indicando que operaciones o capacidades incorpora cada uno

Respuesta:

Dentro del ecosistema de **Spring Data JPA**, la jerarquía de repositorios está diseñada para ofrecer niveles incrementales de funcionalidad, permitiendo que el desarrollador elija la interfaz que mejor se adapte a sus necesidades.

1. CrudRepository

Es el nivel base de la jerarquía funcional y hereda de la interfaz marcadora `Repository`. Su propósito es automatizar las operaciones de persistencia más comunes para una entidad.

- **Capacidades:** Provee operaciones estándar de **CRUD** (Crear, Recuperar, Actualizar y Borrar).
- **Operaciones principales:** Incorpora automáticamente métodos como `save(S entity)`, `findById(ID id)`, `findAll()`, `count()`, `deleteById(ID id)` y `existsById(ID id)`.
- **Uso ideal:** Casos donde solo se requiera una gestión básica de la persistencia sin necesidad de consultas complejas de ordenamiento o paginación.

2. PagingAndSortingRepository

Este nivel hereda de `CrudRepository` y agrega capacidades para el manejo eficiente de grandes volúmenes de datos.

- **Lo que agrega:** Incorpora soporte nativo para **paginación y ordenamiento** mediante el uso de las abstracciones `Pageable` y `Sort`.
- **Operaciones principales:**
 - `findAll(Sort sort)`: Permite recuperar todas las entidades aplicando un criterio de ordenación.
 - `findAll(Pageable pageable)`: Permite recuperar fragmentos de datos (páginas) basándose en el número de página, el tamaño de la misma y el orden deseado.
- **Capacidades:** Permite retornar tipos como `Page<T>` (que incluye el conteo total de elementos) o `Slice<T>` (más liviano, sin conteo total), optimizando la performance en aplicaciones web.

3. JpaRepository

Es la interfaz más completa y específica para tecnologías basadas en JPA (como Hibernate). Hereda de `PagingAndSortingRepository` e incorpora funciones técnicas propias del estándar JPA.

- **Lo que agrega:** Capacidades de control sobre el **Contexto de Persistencia** y operaciones de escritura masiva.
- **Operaciones principales:**
 - **Sincronización (Flushing):** Métodos como `flush()` para forzar la sincronización de cambios en memoria con la base de datos física.
 - **Borrado masivo:** Provee métodos como `deleteInBatch(Iterable<T> entities)`, que ejecutan una sola sentencia SQL de borrado para múltiples registros, mejorando el rendimiento respecto a los borrados individuales.
 - **Persistencia mejorada:** Métodos como `saveAndFlush(T entity)` para persistir y sincronizar en un solo paso.
- **Capacidades:** A diferencia de sus predecesores, muchos de sus métodos (como `findAll()`) retornan una `List` en lugar de un `Iterable`, lo que facilita la

manipulación de los resultados en Java.

En resumen: `CrudRepository` se encarga de lo básico, `PagingAndSortingRepository` añade el control sobre el flujo de datos (páginas/orden), y `JpaRepository` suma herramientas técnicas de bajo nivel para interactuar con el motor de persistencia JPA.

Inciso 9

Crear las interfaces de repositorio para las entidades del modelo extendiendo `CrudRepository`. Indicar los parámetros de tipo correctos (entidad e ID) para cada una:

- `PurchaseRepository`,
- `RouteRepository`,
- `UserRepository`,
- `ServiceRepository`,
- `SupplierRepository` y `ReviewRepository`.